

# GitHub Actions: Execution Control & Error Handling Summary

DevOps Reference Guide

2026

## 1 The Core Behavior Matrix

GitHub Actions processes steps sequentially. The table below outlines how specific configurations alter the workflow's progression.

| Configuration                        | Step Failure Impact     | Subsequent Steps Status                | Final Job Status |
|--------------------------------------|-------------------------|----------------------------------------|------------------|
| Default Behavior                     | Stops job immediately   | All remaining steps are <b>skipped</b> | Failed <b>X</b>  |
| <code>continue-on-error: true</code> | Silences command error  | Following steps <b>run normally</b>    | Success <b>●</b> |
| <code>if: &lt;condition&gt;</code>   | Step runs based on rule | Evaluated step-by-step                 | Conditional      |

## 2 Conditional Gatekeepers (if: Functions)

By default, every step has an invisible conditional attached to it: `if: success()`. Overwriting this property changes how the runner evaluates step entry:

- `if: success()` [Default] Runs only if all previous steps in the job completed without an unhandled error.
- `if: failure()` Evaluates to true if any prior step in the job has crashed. Used for diagnostics, reports, and notifications.
- `if: always()` Forces execution regardless of structural outcomes or manual cancellations. Crucial for resource cleanup.
- `if: cancelled()` Executes exclusively if the pipeline run was explicitly stopped by an operator.

## 3 YAML Deployment Patterns

### 3.1 Pattern A: Target Artifact Uploads on Failure

```
- name: Execute Test Suite
  id: suite_tests
  run: npm run test

- name: Upload Test Report
  # Runs only if the suite_tests step specifically crashes
  if: failure() && steps.suite_tests.outcome == 'failure'
  uses: actions/upload-artifact@v4
  with:
    name: test-report
    path: coverage/
```

## 3.2 Pattern B: Continuing on Non-Critical Errors

```
- name: Experimental Linter
  # If this command exits with a non-zero code, pipeline remains green
  continue-on-error: true
  run: npm run lint:experimental

- name: Production Compile
  run: npm run build
```

## 4 The Architectural Difference: Outcome vs. Conclusion

While both properties track the execution state of a step inside the `steps` context object, they serve completely different structural tiers within GitHub Actions.

| Scenario Code Behavior                                   | outcome Value | conclusion Value | UI Representation    |
|----------------------------------------------------------|---------------|------------------|----------------------|
| Script exits with code 0                                 | 'success'     | 'success'        | Green Checkmark      |
| Script crashes (Normal step)                             | 'failure'     | 'failure'        | Red Cross            |
| Script crashes with <code>continue-on-error: true</code> | 'failure'     | 'success'        | Orange Warning Badge |

### The Execution Difference (Under the Hood)

There is **no technical difference** in how your scripts execute on the actual runner hardware; a crash always returns the exact same exit status code to the operating system shell. The distinction lies entirely in how GitHub's central management platform interprets the data:

- `steps.<step_id>.outcome` [**The Historical Record**]: This is a read-only variable that archives the raw reality of the runner machine. It captures what actually happened. If a script fails, this is permanently stamped as `'failure'`.
- `steps.<step_id>.conclusion` [**The Orchestration Layer**]: This is the variable GitHub uses to calculate pipeline control flow. It updates the UI colors, adjusts the visual badges (Green, Orange, or Red), determines if the entire run passes or fails, and checks whether subsequent steps with default `if: success()` guards are allowed to run.

When you apply `continue-on-error: true`, you are issuing a direct command to the orchestration layer: *“Acknowledge that the raw **outcome** was a failure, but intercept the pipeline logic and force the final **conclusion** to register as a success so execution flows without interruption.”*